DeepLearning

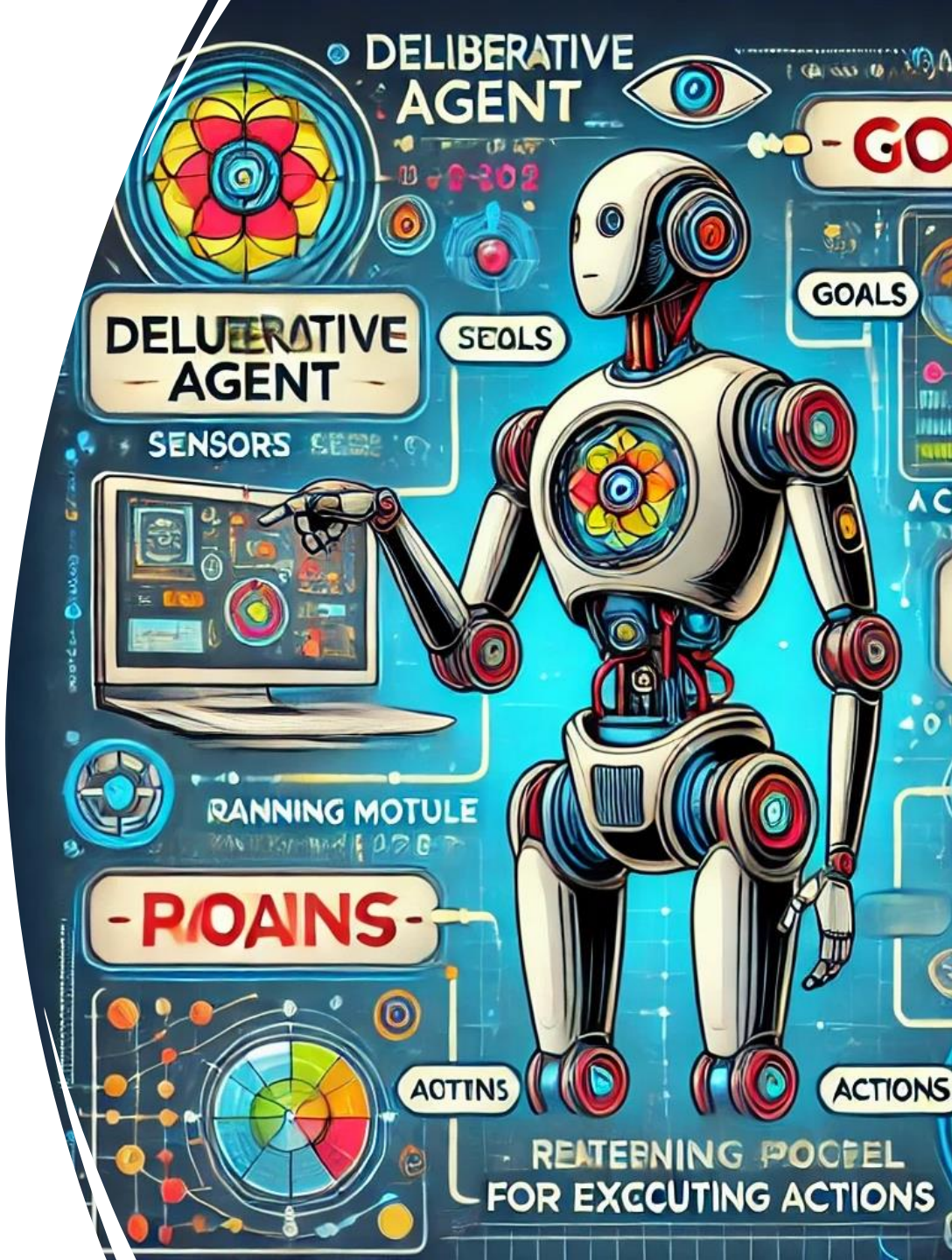
Introducción

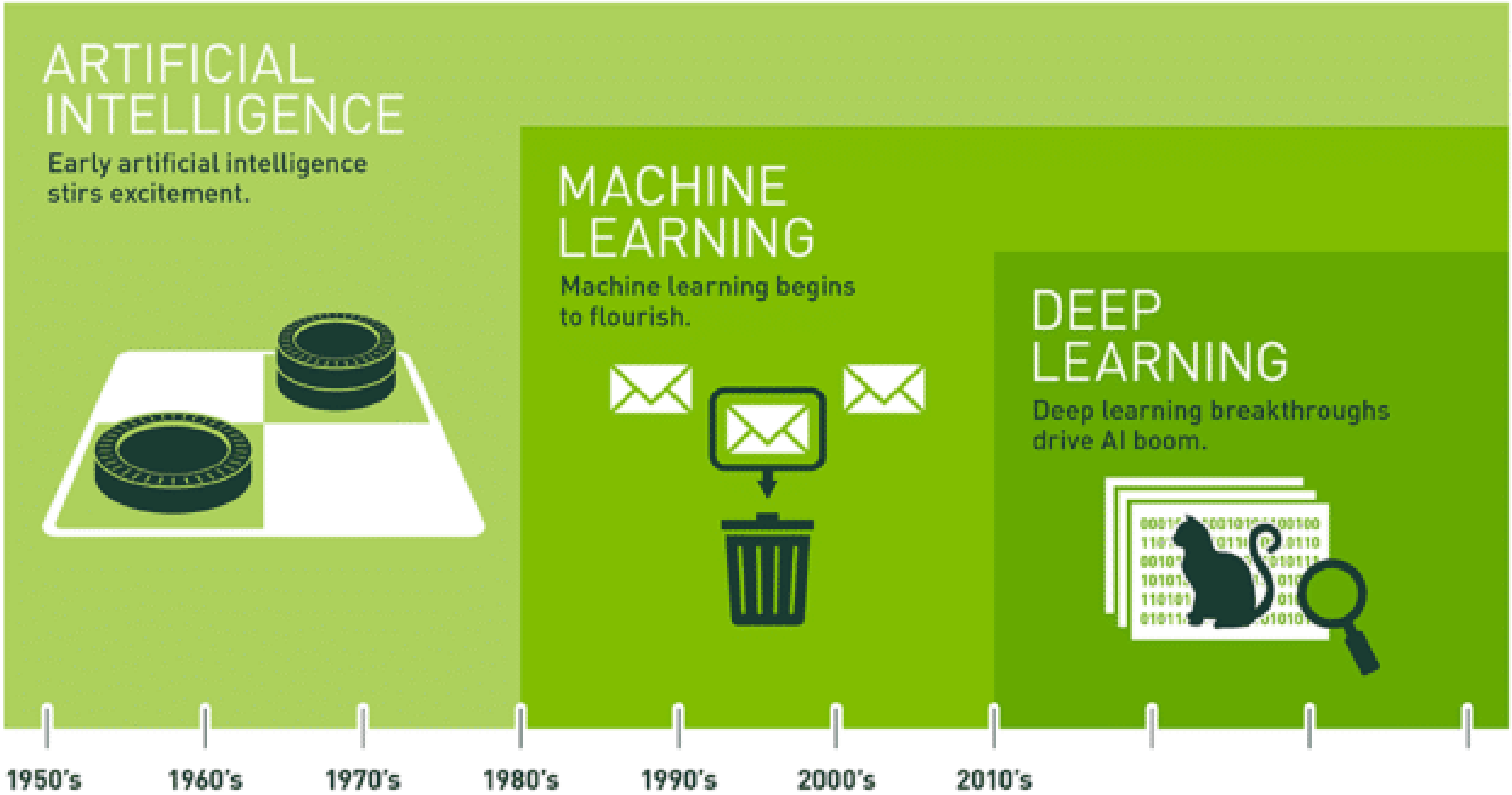
Universidad
Tecnológica
de Bolívar

Inteligencia Artificial

Inteligencia Artificial

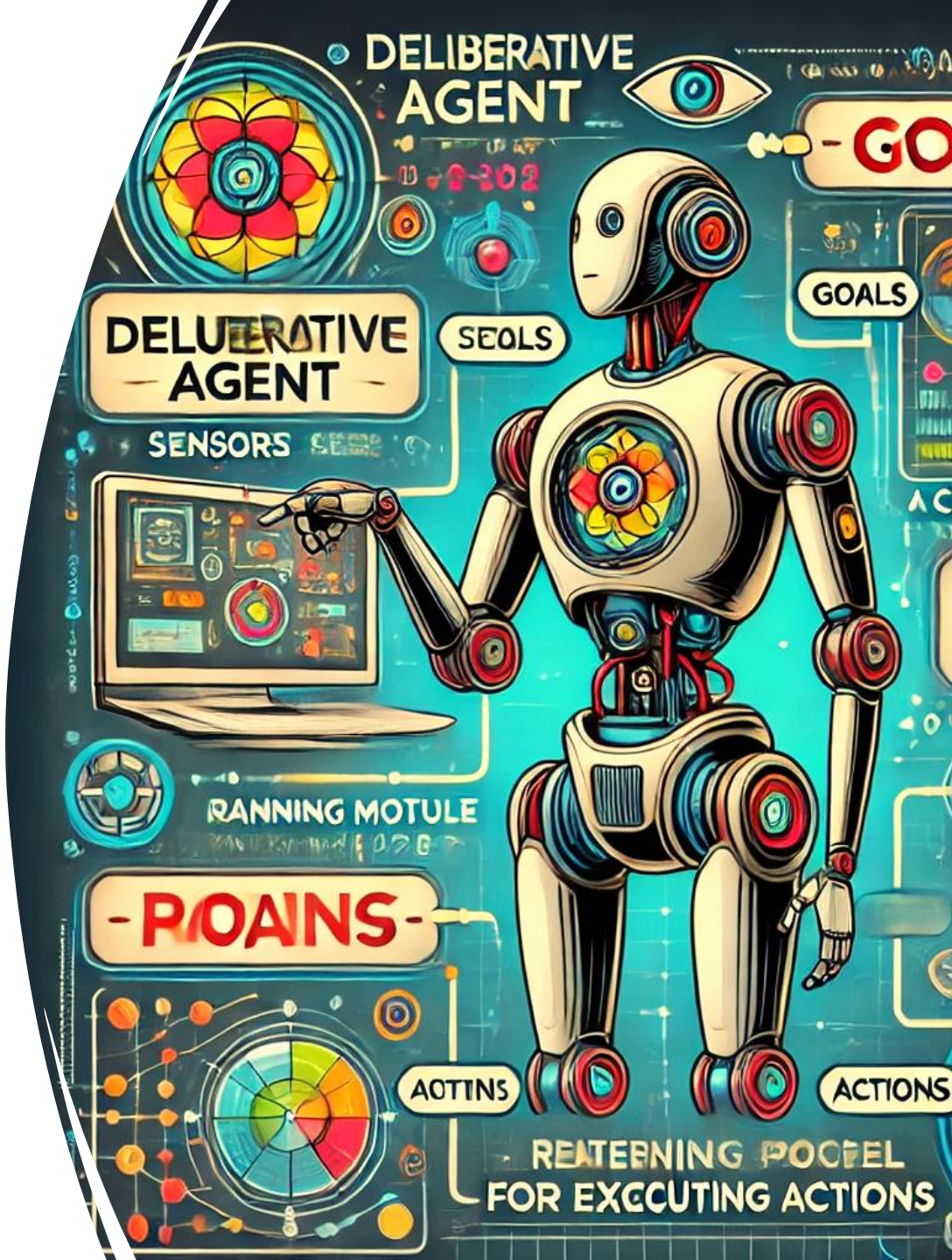
Es la parte de la **Ciencia** que se ocupa del diseño de sistemas de **computación inteligentes**, es decir, sistemas que **exhiben** las **características** que asociamos a la inteligencia en el comportamiento humano que se refiere a la **comprensión** del **lenguaje**, el **aprendizaje**, el **razonamiento**, la **resolución** de **problemas**, entre otros.



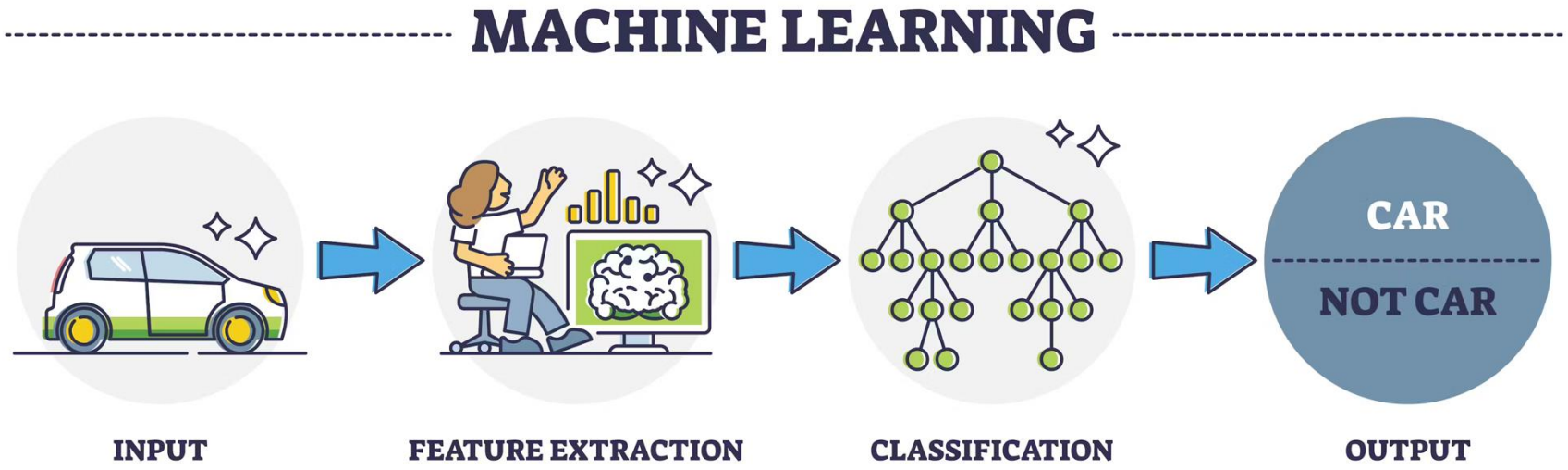

ARTIFICIAL INTELLIGENCE
Early artificial intelligence stirs excitement.
MACHINE LEARNING
Machine learning begins to flourish.
DEEP LEARNING
Deep learning breakthroughs drive AI boom.
1950's
1960's
1970's
1980's
1990's
2000's
2010's

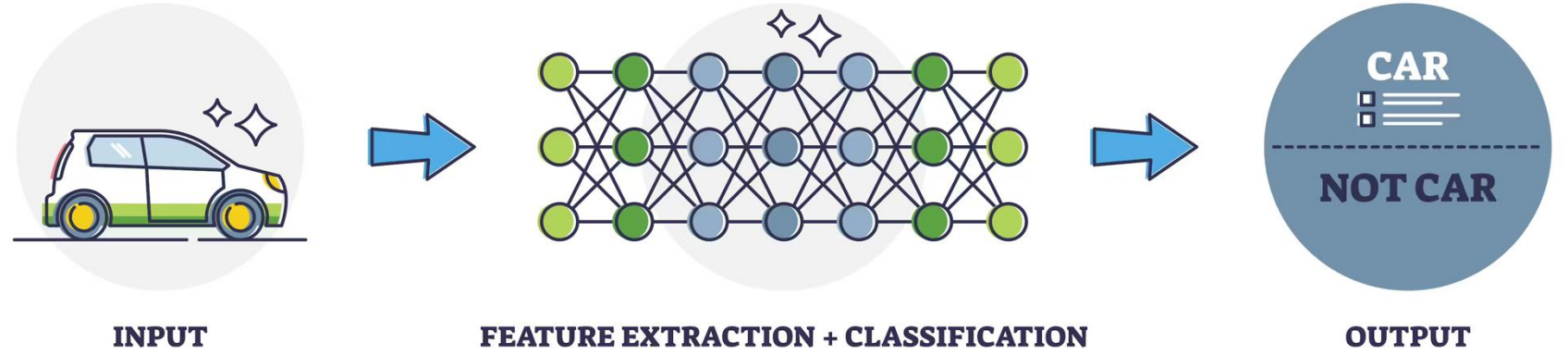
Aprendizaje de Máquina

Es un **subcampo** de las **ciencias de la computación** y una **rama** de la **inteligencia artificial**, cuyo objetivo es desarrollar técnicas que permitan que las computadoras **aprendan**, usando algoritmos y modelos estadísticos. Los algoritmos de aprendizaje automático crean un **modelo matemático basado en datos**, para hacer **predicciones** o **decisiones** sin ser programado explícitamente para realizar la tarea.



ML vs DeepL

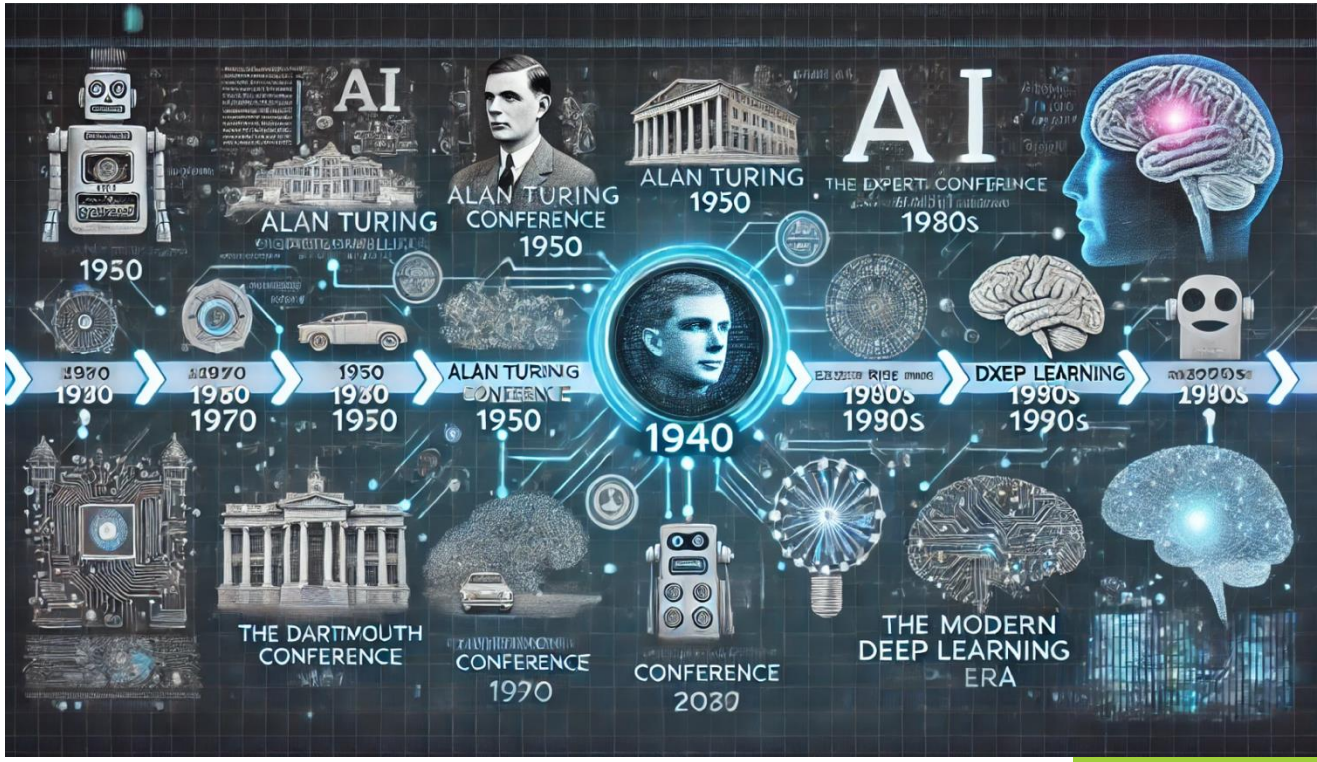

MACHINE LEARNING
CAR
NOT CAR
INPUT
FEATURE EXTRACTION
CLASSIFICATION
OUTPUT


DEEP LEARNING
CAR
NOT CAR
INPUT
FEATURE EXTRACTION + CLASSIFICATION
OUTPUT

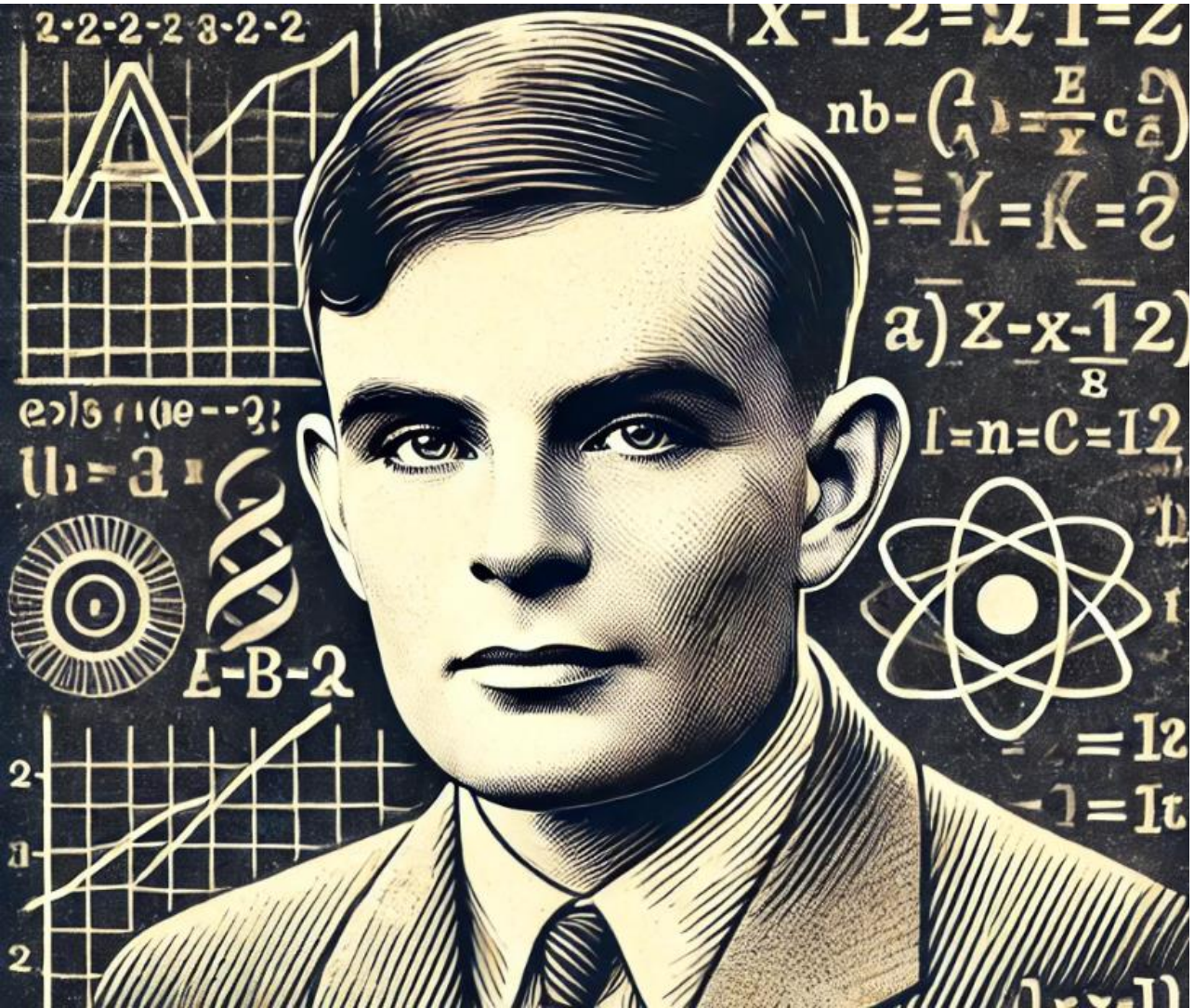
¿Qué es la Inteligencia Artificial?

La IA busca crear sistemas capaces de realizar tareas que requieren inteligencia humana.

- IA Débil: Diseñada para tareas específicas (Ej: Siri, ChatGPT).
- IA Fuerte: Capaz de razonar como un humano (aún en desarrollo).



Primeros Pasos de la IA (1940-1950)



- Alan Turing (1950): 'Computing Machinery and Intelligence' y la Prueba de Turing.
- McCulloch y Pitts (1943): Primer modelo matemático de una neurona artificial.
- Norbert Wiener y la Cibernética: Comunicación en máquinas y seres vivos.

Auge y Primeros Sistemas de IA (1950-1970)

- Dartmouth Conference (1956): Nacimiento de la IA como disciplina.
- Frank Rosenblatt (1958): Perceptrón, primera red neuronal con aprendizaje.
- Lenguajes de IA: LISP (1958) y Prolog (1972).

Primer Invierno de la IA (1970-1980)

- Limitaciones tecnológicas y falta de poder computacional.
- Minsky y Papert (1969): Demuestran que el Perceptrón no puede resolver problemas no lineales.
- Disminución de la financiación y estancamiento de la IA.

Resurgimiento con Sistemas Expertos (1980-1990)

- Modelos basados en reglas aplicados en diagnóstico médico y negocios.
- Ejemplo: MYCIN para diagnóstico de infecciones en sangre.
- Inversión renovada en IA.

Segundo Invierno de la IA (1990-2000)

- Problemas con los sistemas expertos: altos costos y escalabilidad limitada.
- Auge del Machine Learning basado en estadísticas y datos.

El Renacimiento del Deep Learning (2000-presente)

- Factores clave: Big Data, avances en GPUs y algoritmos eficientes.
- Casos de éxito:
  - AlexNet (2012) arquitectura de red neuronal convolucional (CNN). Tenía 60 millones de parámetros y 650.000 neuronas,
  - AlphaGo (2016), tienen 13 capas ocultas, cada una con unas 260 mil neuronas y unos 2,7 millones de pesos
  - ChatGPT (2022), GPT-1, tiene 120 millones de parámetros. GPT-2 tiene 1.500 millones de parámetros, mientras que GPT-3 tiene 175.000 millones.
  - DeepSeeker (2025), proporciona 671.000 millones de parámetros y activa 37.000 millones de parámetros para cada token.

¿Por qué ahora?

Democratización del Conocimiento y los Recursos

- Democratización de la IA - **software abierto**
- Lenguajes especializados abiertos: **Python**, R
- Algoritmos avanzados abiertos: LightGBM (Microsoft), XGBoost, Catboost (Yandex)
- Redes neuronales: **Tensorflow** (Google), **PyTorch** (usado por Facebook AI Research Group, IBM, Yandex), **Scikit Learn**
- Infraestructuras big data: **Hadoop**, Spark
- Cloud y sus herramientas: **Microsoft** Azure, **Amazon** Web Services, **Google** Cloud

Procesamiento

- **CPUs** cada vez más avanzados (incluso opción de alquilar en la nube). ¡En los últimos 25 años la computación ha aumentado su velocidad en 1 millón de veces ! Y sigue creciendo
- Evolución de las **GPUs** para las redes neuronales, impulsadas por la industria del videojuego y el minado de bitcoin. Hoy por algo más de 1000 euros se puede tener una GPU con 30 TFlops, algo impensable hace pocos años (1997: 30000 USD/GFlops -> 2019: 0.03 USD/GFlops)

Más Capacidad

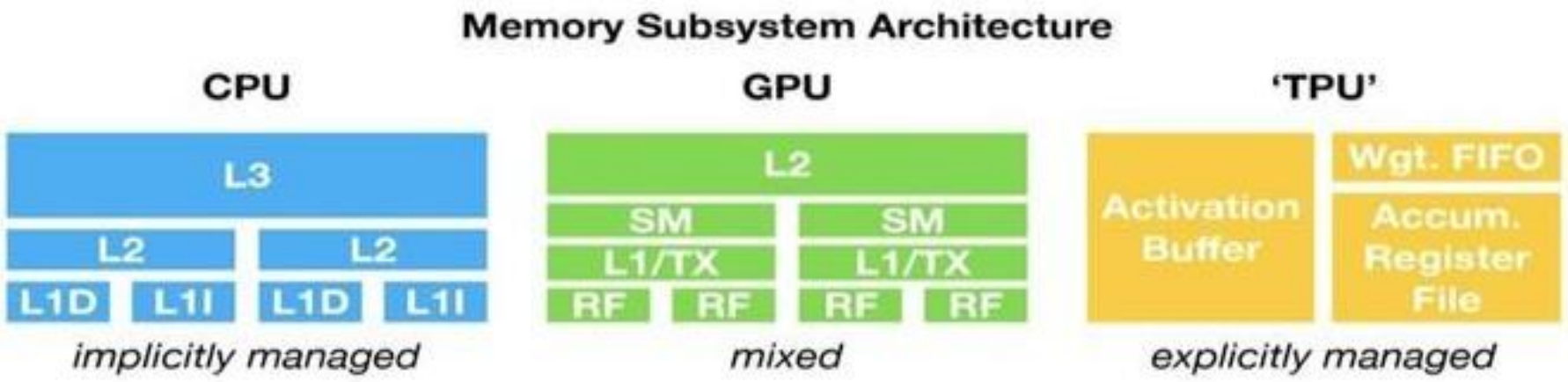
- **NPU** (Unidad de Procesamiento de Redes Neuronales) está diseñado específicamente para acelerar aplicaciones de IA, a través de procesadores que imitan el sistema neural humano.

- **TPU** (Unidad de Procesamiento Tensorial) es un procesador desarrollado por Google específicamente para acelerar tareas de aprendizaje. A diferencia de las GPUs, los TPUs están diseñados para cálculos de baja precisión a gran escala. En tareas de inferencia de IA utilizando redes neuronales, el rendimiento de los TPU es de 15 a 30 veces el de las GPUs y CPUs contemporáneas.

CPU vs GPU vs TPU

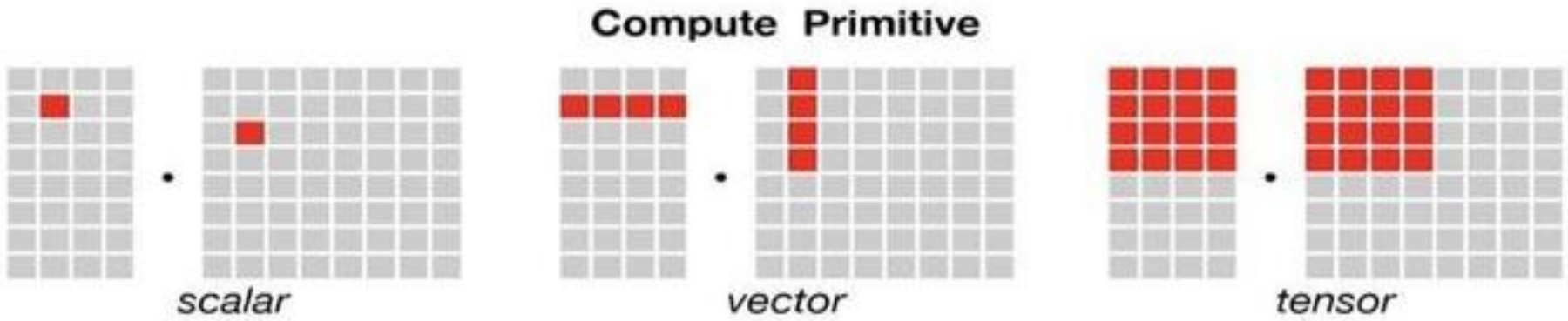
Memory Subsystem Architecture

This image captures the memory architecture of CPU, GPU and TPU:



Compute Primitive

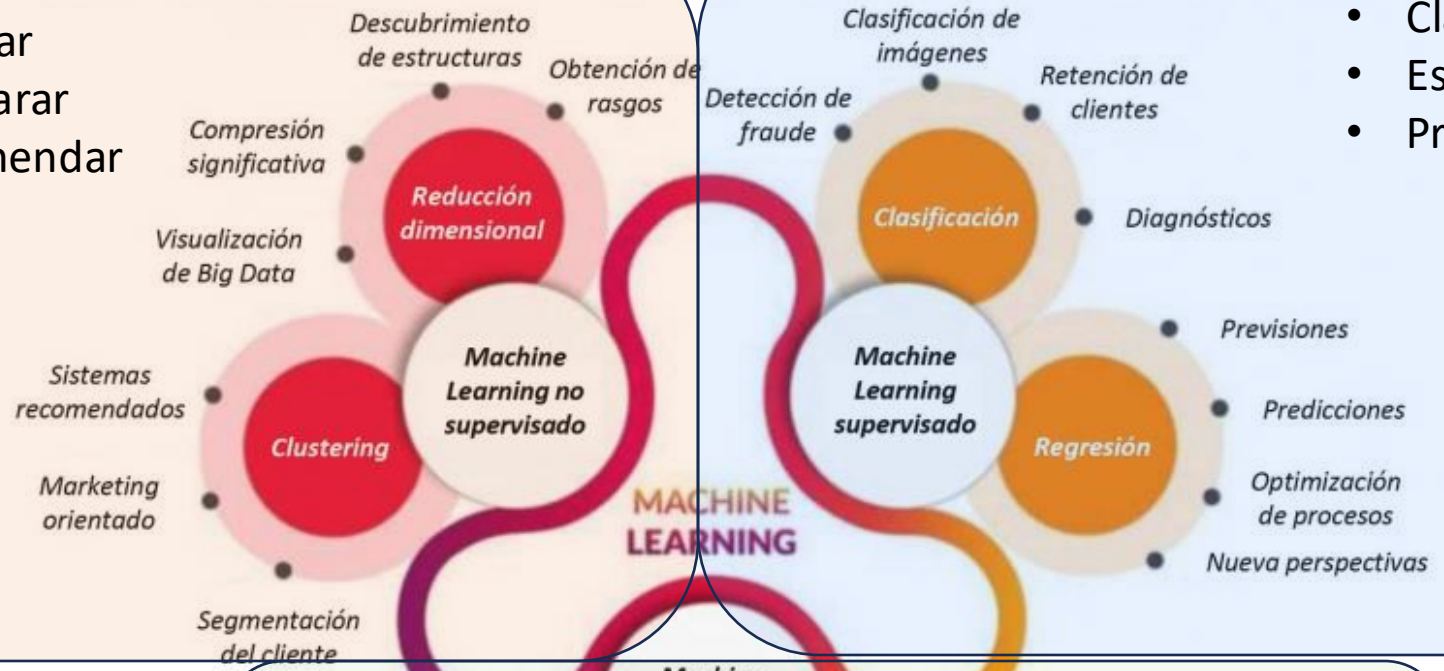
This image summarizes the compute primitive (smallest unit) in CPU, GPU and TPU:



Clasificación de Machine Learning

- Agrupar
- Comparar
- Recomendar

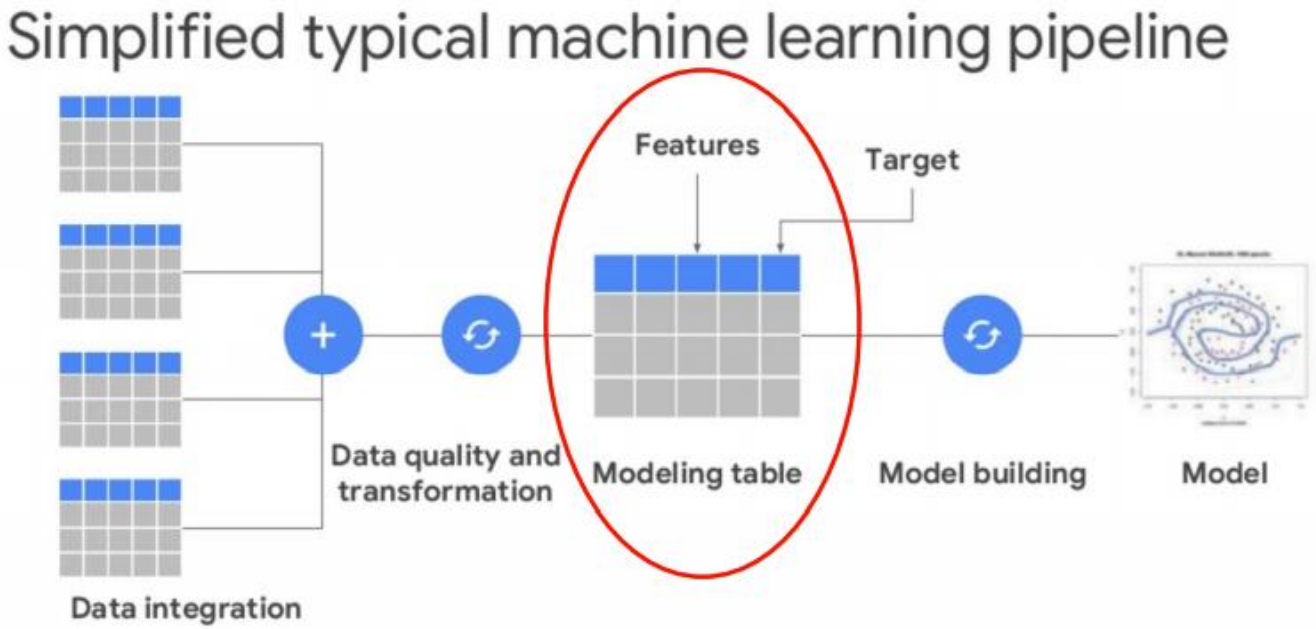
- Detectar
- Clasificar
- Estimar
- Predecir



- Maximizar Recompensa
- Entrenar

Recordemos

Entrenamiento



Origen:

- Bases de Datos
- Datos de Campo

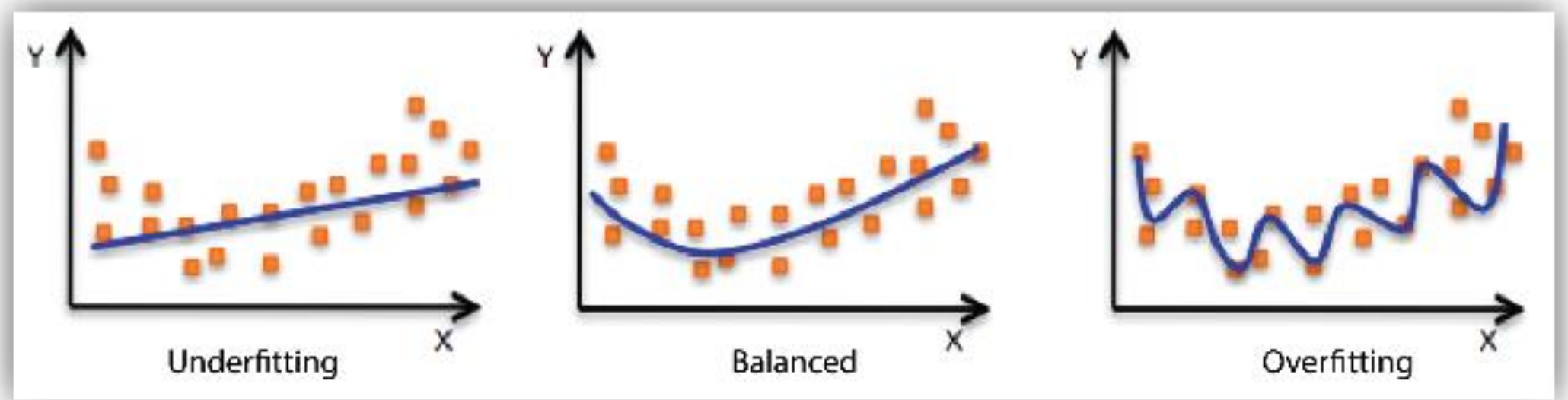
Tipos de datos:

- Datos temporales
- Datos estáticos

Procesamiento:

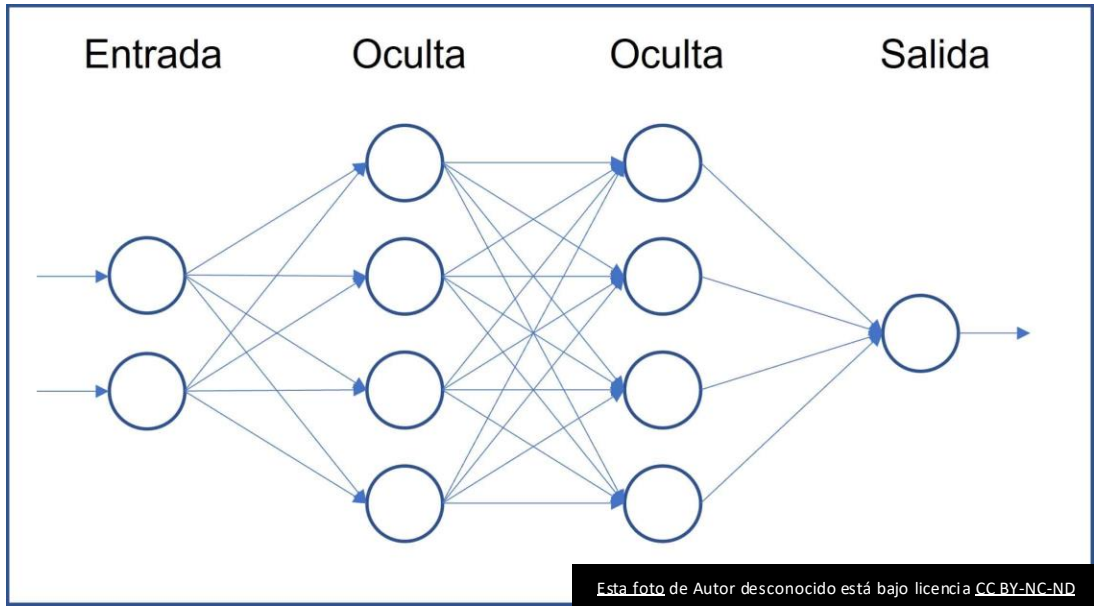
- Organizar los datos en una tabla de registros, en los que cada fila representa una instancia de aprendizaje, y las columnas representan las características (variables o "**features**")
- Incorporar la variable destino ("**target**") con el valor de cada instancia, que es lo que se pretende aprender

Ajuste del Modelo



Redes Neuronales

Introducción a Redes Neuronales

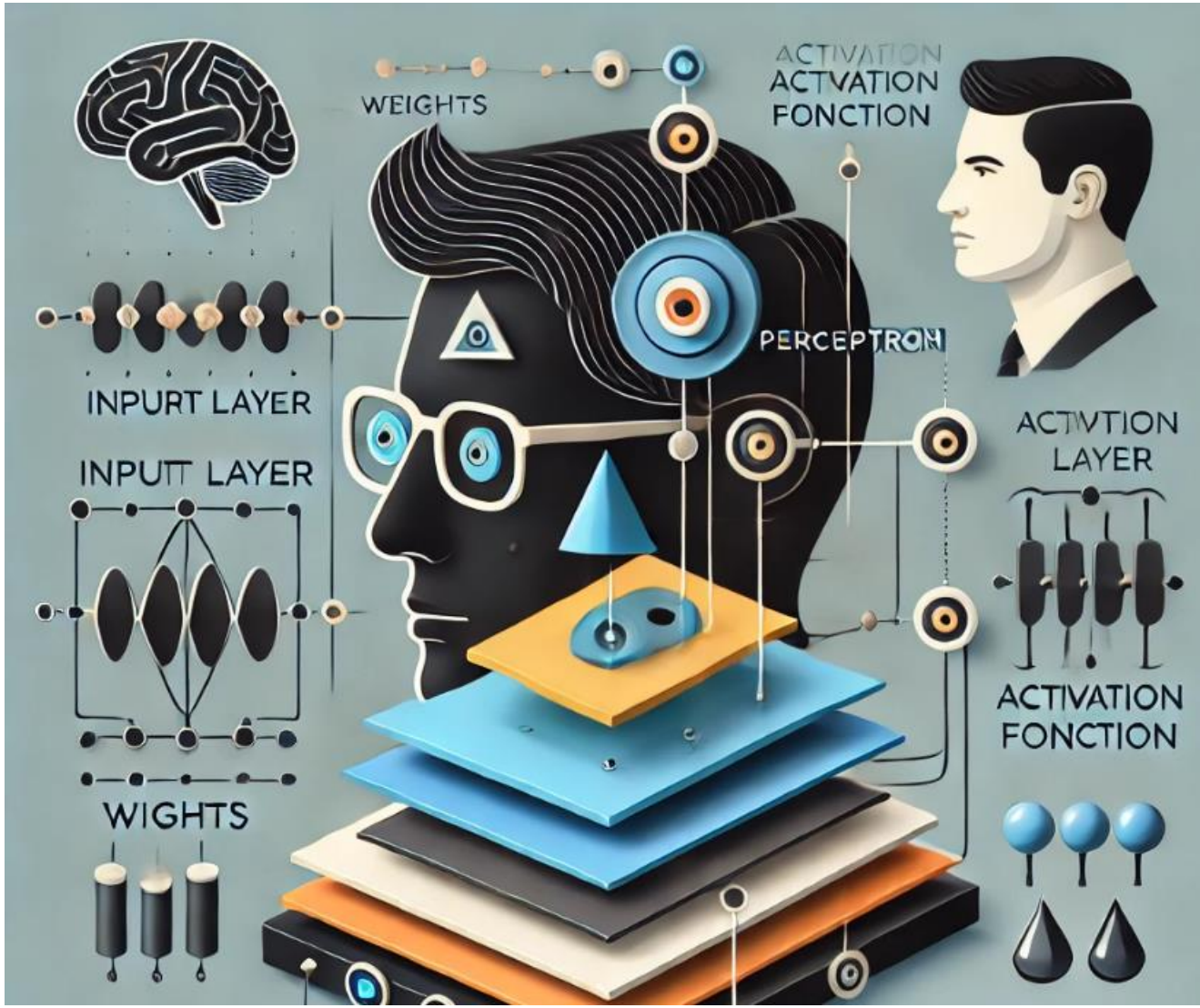


Esta foto de Autor desconocido está bajo licencia CC BY-NC-ND

- Inspiración en el cerebro humano.
- Redes neuronales artificiales: Combinación de neuronas artificiales.

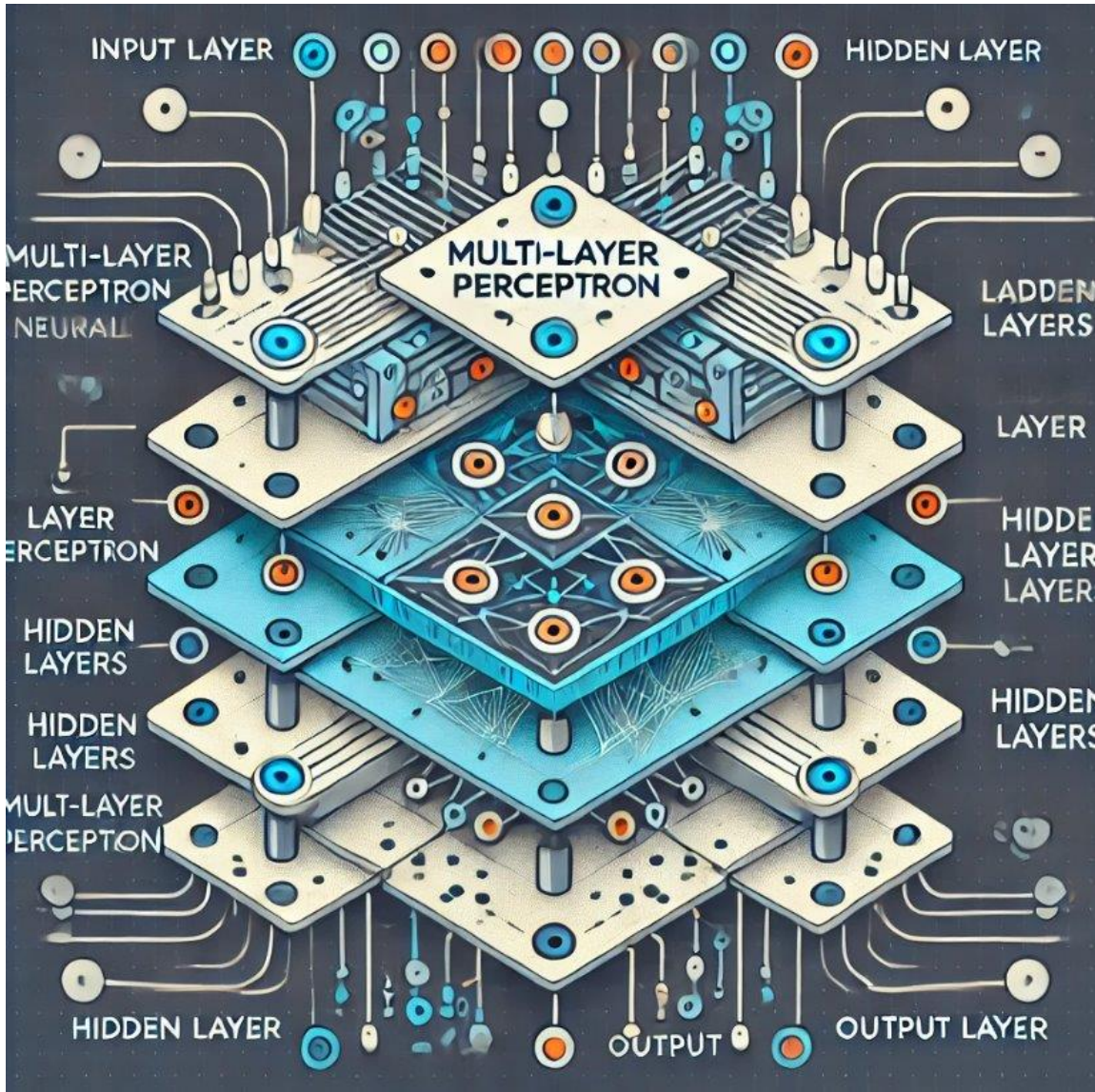
Perceptrón (1958)

- Primer modelo de red neuronal.
- Algoritmo que imita el funcionamiento de las neuronas del cerebro y se utiliza para clasificar datos
- Aprende ajustando pesos mediante reglas simples.
- No resuelve problemas no lineales.



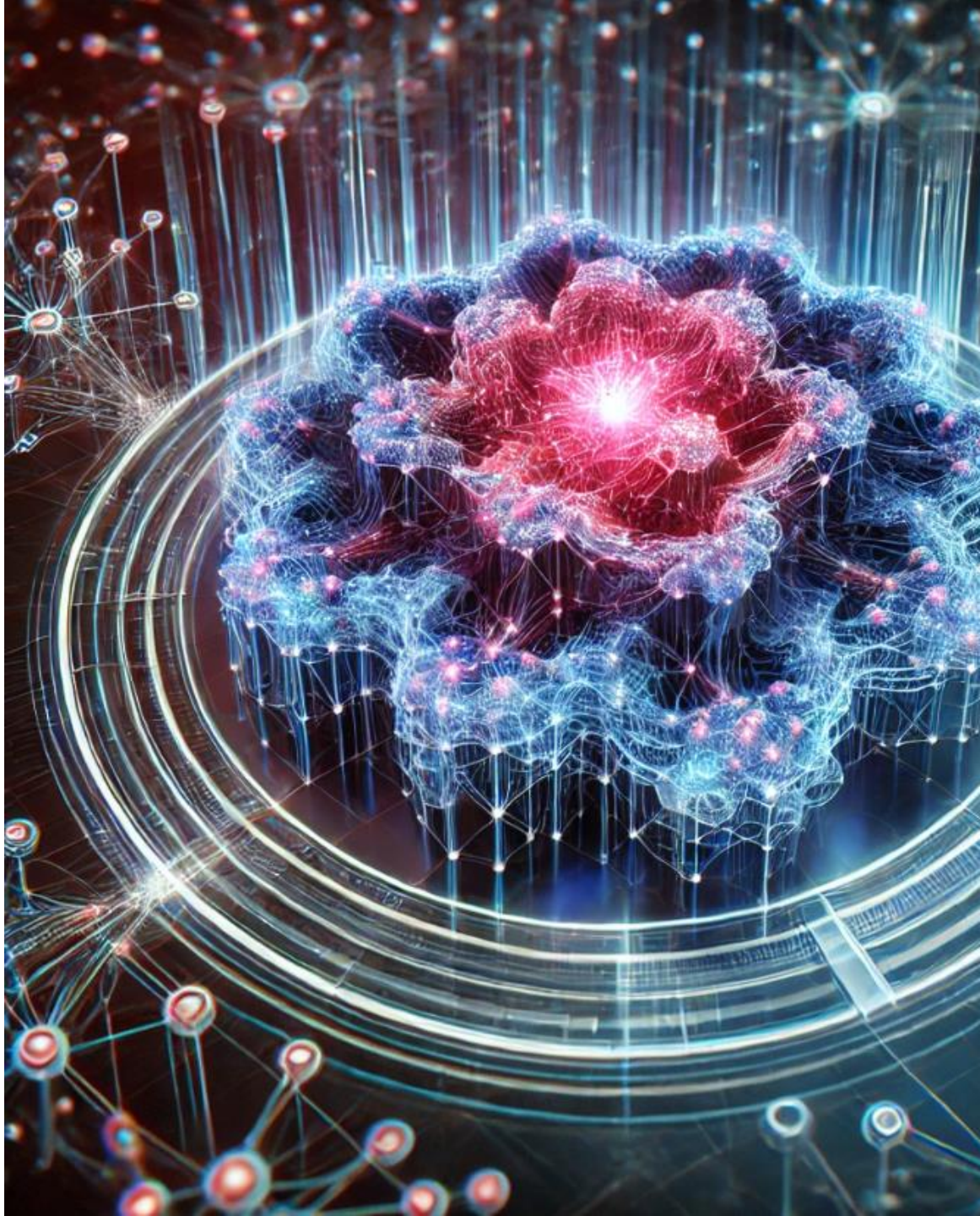
Redes Neuronales Multicapa (MLP)

- Introducción de capas ocultas para mejorar el aprendizaje.
- Backpropagation (1986): Método para ajustar pesos.
- Funciones de activación: Sigmoide, ReLU, Tanh.



Deep Learning

El deep learning es un subconjunto del machine learning que **utiliza redes neuronales multicapa**, llamadas redes neuronales profundas, para simular el complejo poder de toma de decisiones del cerebro humano.



1 Modelos de DeepLearning

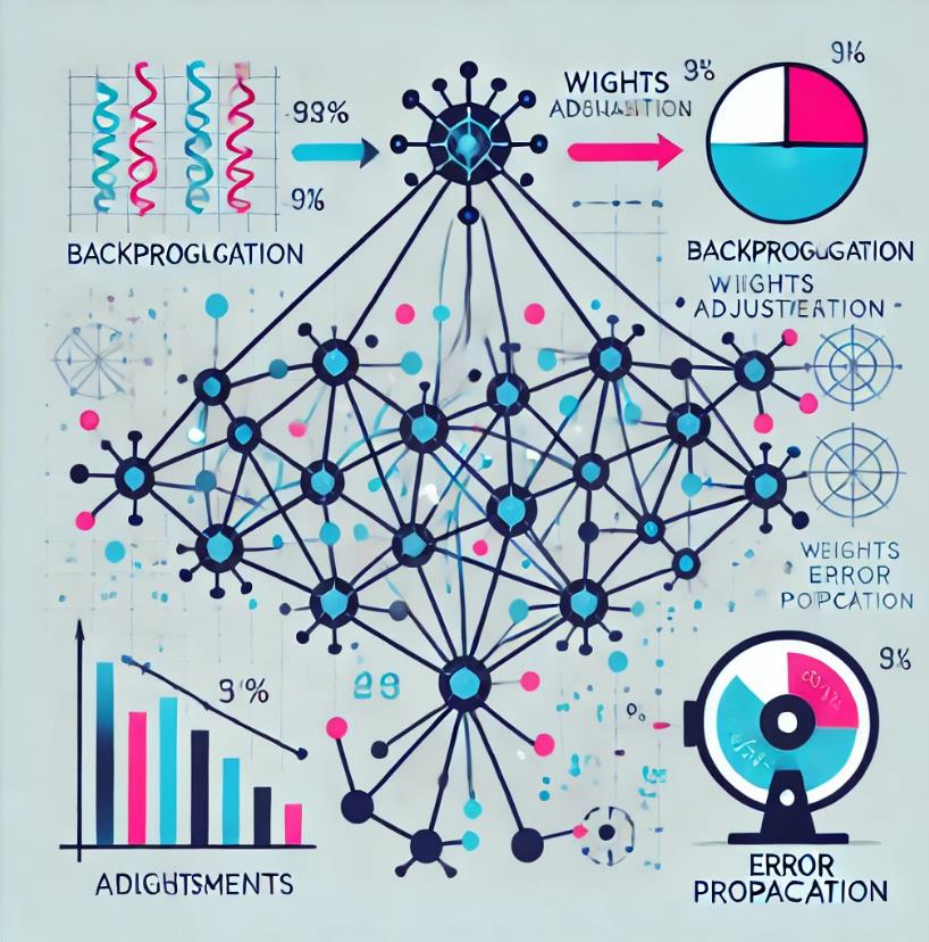
- **Redes Neuronales Feedforward (FNN)** Son las redes neuronales más simples, donde los datos fluyen en una sola dirección: de la entrada a la salida. No hay retroalimentación o conexiones cíclicas.
  - Aplicaciones comunes: Clasificación básica, regresión
  - Ejemplo típico: Clasificación de dígitos (MNIST)
- **Redes Neuronales Convolucionales (CNN)** Las **CNNs** son redes especializadas en el procesamiento de datos con una estructura en cuadrícula (como las imágenes). Utilizan filtros convolucionales que detectan patrones locales, como bordes, texturas, etc.
  - Aplicaciones comunes: Visión por computadora, reconocimiento de imágenes
  - Ejemplo típico: Clasificación de objetos en imágenes
- **Redes Neuronales Recurrentes (RNN)** Las **RNNs** están diseñadas para procesar datos secuenciales, como texto o series temporales, y tienen conexiones hacia atrás que permiten que la salida de una capa dependa de su entrada anterior.
  - Aplicaciones comunes: Procesamiento de secuencias, series temporales
  - Ejemplo típico: Predicción de la siguiente palabra
- **LSTM y GRU** Son variantes de las RNNs diseñadas para resolver el problema del **desvanecimiento del gradiente**. Las **LSTM** y **GRU** son más efectivas en el manejo de dependencias a largo plazo en secuencias.
  - Aplicaciones comunes: Modelado de secuencias largas
  - Ejemplo típico: Traducción de texto, análisis de sentimientos
- **Redes Generativas Antagónicas (GANs)** Las **GANs** constan de dos redes neuronales: el **generador** y el **discriminador**. El generador crea datos falsos (por ejemplo, imágenes), mientras que el discriminador intenta identificar si esos datos son reales o generados.
  - Aplicaciones comunes: Generación de datos, imágenes sintéticas
  - Ejemplo típico: Generación de rostros humanos

2 Modelos de DeepLearning.

- **Transformers (y mecanismos de atención)** El **mecanismo de atención** y el modelo **Transformer** son arquitecturas avanzadas que mejoran la capacidad de las redes para enfocarse en partes importantes de la entrada, sin depender de la secuencia ordenada como las RNNs. Son especialmente eficaces para tareas que requieren considerar todo el contexto simultáneamente.
  - Aplicaciones comunes: Procesamiento de lenguaje natural
  - Ejemplo típico: Traducción automática, GPT
- **Autoencoders** Los **autoencoders** son redes neuronales utilizadas para **reducción de dimensionalidad** o **aprendizaje no supervisado**. Un autoencoder intenta aprender una representación comprimida de los datos (código) y luego reconstruir la entrada a partir de esa representación.
  - Aplicaciones comunes: Reducción de dimensionalidad, compresión de datos
  - Ejemplo típico: Detección de anomalías, reducción de ruido
- **Redes Neuronales de Procesamiento de Grafos (GNN)** Las **GNNs** son diseñadas para trabajar con datos que pueden representarse como grafos, donde las relaciones entre nodos son fundamentales. Estas redes se utilizan para modelar datos con estructuras de grafo como redes sociales, moléculas, etc.
  - Aplicaciones comunes: Análisis de redes, estructuras de grafo
  - Ejemplo típico: Predicción de enlaces en redes sociales
- **Capsule Networks** intentan mejorar las limitaciones de las CNNs al capturar las relaciones espaciales entre objetos en una imagen. Usan "cápsulas" (un conjunto de neuronas) para representar características y su orientación, permitiendo una mejor generalización en tareas de visión por computadora.
  - Aplicaciones comunes: Reconocimiento de patrones espaciales complejos
  - Ejemplo típico: Reconocimiento de objetos en imágenes complejas

Entrenamiento

Aprendizaje supervisado con optimización de pesos



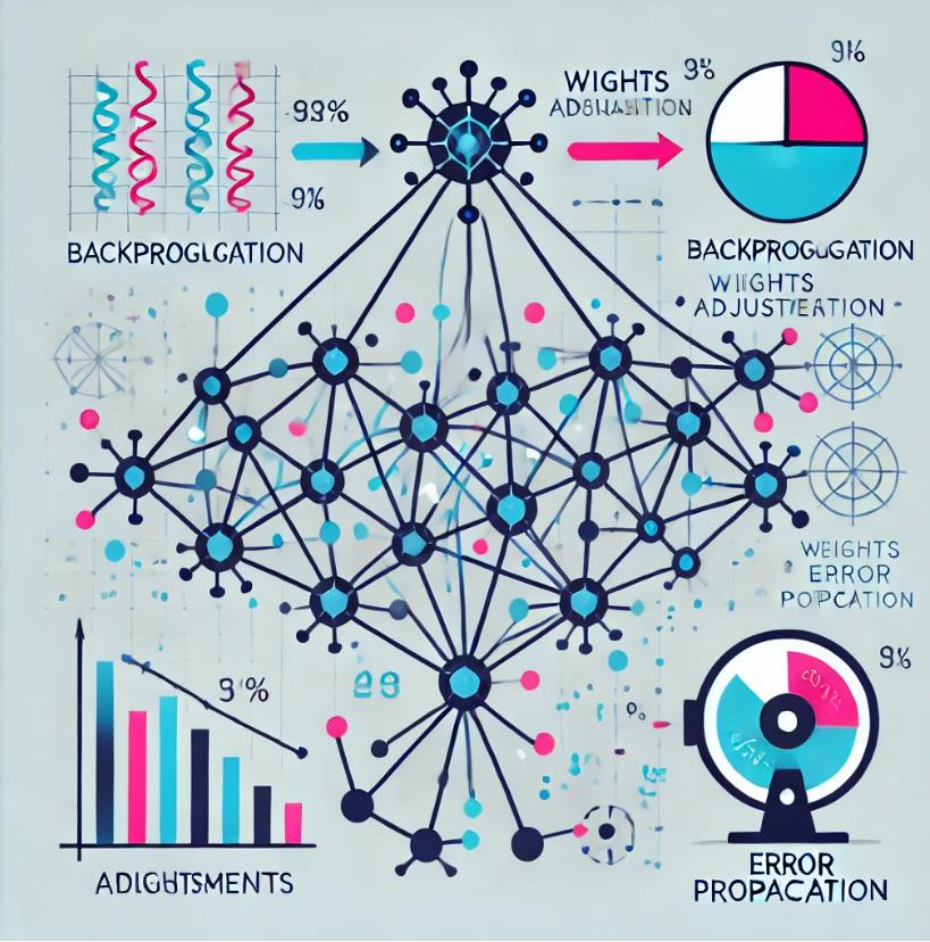
El **aprendizaje supervisado** es un tipo de entrenamiento en el que alimentamos a la red neuronal con un conjunto de datos etiquetado, es decir, tenemos tanto las entradas como las salidas esperadas. La red intenta aprender a mapear las entradas a las salidas correctas.

- En cada iteración del proceso de entrenamiento, la red hace una predicción, y luego ajusta sus **pesos** (los parámetros que controlan cómo se transmiten los datos a través de la red) para que la predicción se acerque más a la salida esperada.
- **Optimización de pesos**: Es el proceso de ajustar esos pesos a través de un algoritmo que busca minimizar el error entre la salida predicha y la salida real. El algoritmo más común es el **descenso de gradiente**, que se utiliza para encontrar el conjunto de pesos que reduce al máximo el error.

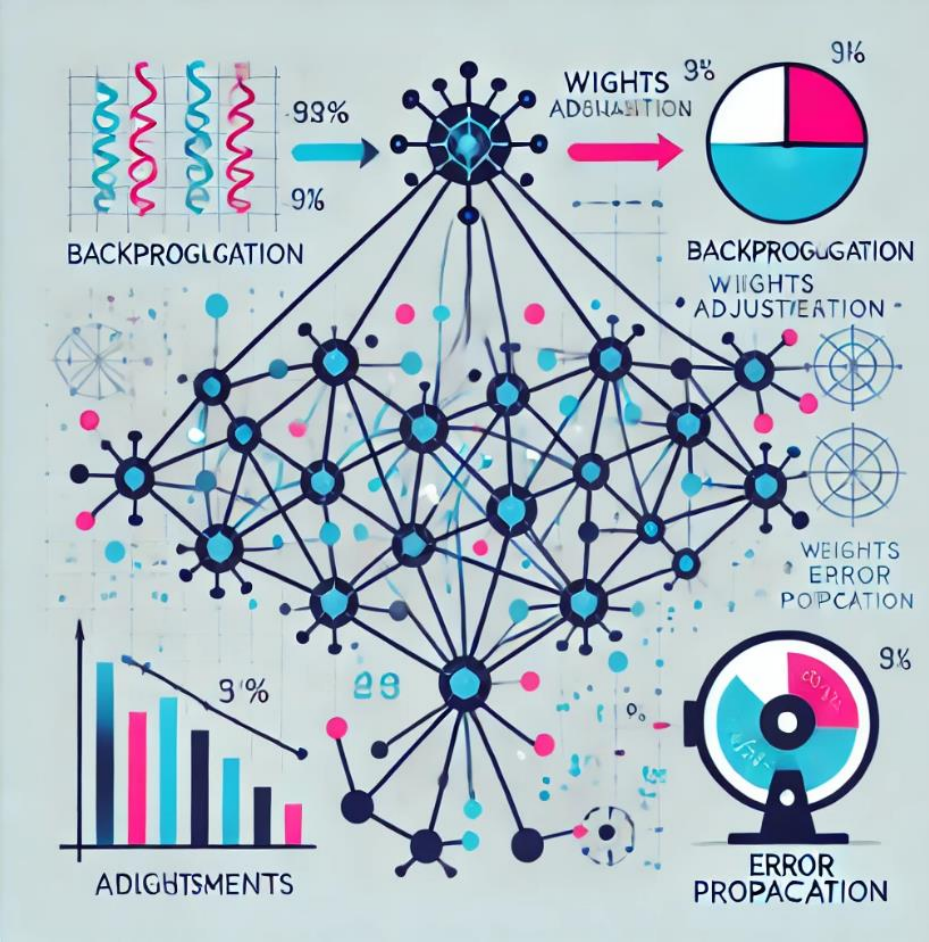
Un ejemplo de esto es cuando entrenas un modelo para reconocer imágenes de gatos y perros. El modelo ajustará sus pesos después de cada imagen que procesa, mejorando gradualmente para clasificar correctamente en función de las características de la imagen.

Función de costo y descenso de gradiente

- **Función de costo** (o función de pérdida) mide cuán lejos están las predicciones del modelo respecto a las salidas esperadas. Una de las funciones de costo más comunes es la entropía cruzada (para clasificación) o el error cuadrático medio (para regresión).
  - Ejemplo: Si un modelo predice que la probabilidad de que una imagen sea un gato es 0.2, pero la etiqueta real es 1 (es un gato), la función de costo calcula qué tan grande es ese error.
- **Descenso de gradiente** es el proceso de minimizar esa función de costo. El algoritmo calcula el gradiente (la pendiente) de la función de costo con respecto a los pesos. Básicamente, es como un GPS que nos dice hacia dónde debemos movernos para reducir el error.
  - Este proceso se realiza de manera iterativa: calculando los gradientes y actualizando los pesos en la dirección opuesta (para minimizar el costo).
  - Si los gradientes son muy grandes o muy pequeños, pueden surgir problemas, como el desvanecimiento del gradiente, que explicaré más adelante.



Problemas comunes



- **Sobreajuste** (overfitting): El sobreajuste ocurre cuando el modelo se ajusta demasiado bien a los datos de entrenamiento, hasta el punto de que captura incluso el ruido y las fluctuaciones aleatorias en esos datos, en lugar de aprender patrones generales. Como resultado, el modelo tiene un rendimiento pobre cuando se enfrenta a nuevos datos.
  - Cómo prevenirlo: Técnicas como la regularización (ej. L1, L2), el dropout (desactivar aleatoriamente neuronas durante el entrenamiento), y la validación cruzada pueden ayudar a reducir el sobreajuste. También es útil usar más datos de entrenamiento o simplificar la red.
- **Desvanecimiento del gradiente**: El desvanecimiento del gradiente ocurre principalmente en redes profundas, donde los gradientes que se calculan durante el descenso de gradiente son tan pequeños que no pueden actualizar efectivamente los pesos de las capas más profundas de la red. Esto hace que el entrenamiento se vuelva muy lento o incluso que la red no aprenda nada.
  - Cómo prevenirlo: Técnicas como la inicialización adecuada de pesos, el uso de funciones de activación como ReLU (que no sufren tanto del problema) o el uso de optimizadores avanzados como Adam pueden mitigar el problema del desvanecimiento del gradiente.

Librerías y Frameworks

TensorFlow

- **Descripción:** Desarrollado por Google, TensorFlow es una biblioteca de código abierto ampliamente utilizada para tareas de machine learning y deep learning. Ofrece flexibilidad y escalabilidad, permitiendo la construcción de redes neuronales complejas y su implementación en diversas plataformas.
- **Características destacadas:**
  - Manejo eficiente de grandes volúmenes de datos y modelos complejos.
  - Facilidad de implementación en entornos de producción, incluyendo web, dispositivos móviles y sistemas distribuidos.
  - Amplia comunidad de usuarios y abundante documentación.

ThePhoto de PhotoAuthor se concede bajo licencia de CCYYSA.

PyTorch

Descripción: Desarrollado por Facebook, PyTorch es un framework de código abierto que ha ganado popularidad por su facilidad de uso y flexibilidad. Es especialmente valorado en la comunidad investigadora debido a su enfoque intuitivo y dinámico.

Características destacadas:

- Ejecución imperativa (define-by-run), lo que facilita la depuración y el desarrollo iterativo.
- Integración sencilla con bibliotecas de Python y soporte para GPU.
- Amplia adopción en la investigación académica y creciente uso en aplicaciones industriales.

PyTorch

ThePhoto de PhotoAuthor se concede bajo licencia de CCYYSA.

Apache MXNet

Descripción: Un proyecto de la Fundación Apache, MXNet es un framework de deep learning flexible y escalable, conocido por su eficiencia y soporte para múltiples lenguajes de programación.

Características destacadas:

- Soporte para programación imperativa y simbólica.
- Optimizado para hardware de alto rendimiento y entornos de producción.
- Utilizado por Amazon como backend para su servicio de deep learning.

Universidad Tecnológica de Bolívar

Gracias

Copyright © 2023. Todos los derechos reservados Universidad Tecnológica de Bolívar.